

category: dotnet-general

tags: [process-start, git-bash, wsl, path-detection, cross-platform]

severity: high

created: 2026-07-24

updated: 2026-07-24

project-origin: LinuxDeployTool

Tự động dò `bash.exe` qua PATH có thể trùng nhầm symlink WSL launcher trong `WindowsApps`, không phải Git Bash/MSYS thật

Tình huống gặp phải

| Đang làm gì? Tính năng gì? Môi trường nào?

Kztek.LinuxDeployTool (app .NET/WinForms) cần tự động phát hiện đường dẫn `bash.exe` của Git for Windows để chạy `Process.Start` gọi `bash scripts/linux-deb/build-deb.sh <version>` cho từng solution. Logic detect: (1) thử vài hardcoded path chuẩn (`C:\Program Files\Git\bin\bash.exe`, (x86) tương tự), (2) nếu không thấy, fallback quét từng thư mục trong biến môi trường `PATH`, tìm file `bash.exe` đầu tiên tồn tại.

Triệu chứng / Lỗi

Trên máy dev thực tế, Git for Windows được cài ở đường dẫn TÙY CHỈNH `C:\Program Files\SetUp\Git\` (không phải path mặc định) — cả 2 hardcoded path đều không khớp. Fallback quét `PATH`: máy có bật WSL nên `%LOCALAPPDATA%\Microsoft\WindowsApps\bash.exe` tồn tại VÀ đứng trước path thật của Git Bash trong `PATH` (vì Git chỉ có `<GitRoot>\cmd` trong `PATH` — chứa `git.exe` — chứ KHÔNG có `<GitRoot>\bin` hay `<GitRoot>\usr\bin`, nơi `bash.exe` thật nằm). Kết quả: fallback trả về `WindowsApps\bash.exe`.

Khi chạy lệnh convert path kiểu MSYS (`E:\foo\bar` → `/e/foo/bar`) rồi gọi process đó, lỗi: `cd: /e/KZTEK/Code_Git/.../iPGSv4: No such file or directory` — dù thư mục đó THẬT SỰ tồn tại trên máy.

Nguyên nhân gốc rễ (Root Cause)

`%LOCALAPPDATA%\Microsoft\WindowsApps\bash.exe` là launcher stub CỦA WINDOWS cho tính năng "Bash on Ubuntu" / WSL — khi hệ thống có bật Windows Subsystem for Linux, file này là **symbolic link trở thẳng tới `wsl.exe`** (`...WindowsApps\MicrosoftCorporationII.WindowsSubsystemForLinux_...wsl.exe`), KHÔNG phải Git Bash/MSYS2. WSL dùng convention mount khác hoàn toàn: ổ đĩa Windows nằm ở `/mnt/e/...`, KHÔNG phải `/e/...` (convention riêng của MSYS2/Git Bash). Code convert path theo chuẩn MSYS (`/e/...`) rồi lỡ chạy qua `wsl.exe` → path không tồn tại trong filesystem WSL → lỗi "No such file or directory", dễ khiến người debug tưởng lỗi nằm ở việc convert path (nhưng convert path đúng, chỉ là process chạy SAI RUNTIME).

Giải pháp

KHÔNG chỉ quét PATH tìm file tên `bash.exe` bất kỳ — phải: (1) ưu tiên derive từ vị trí `git.exe` (luôn đáng tin cậy hơn vì `git.exe` gần như chắc chắn có trong PATH bất kể Git cài ở đâu), dò lên vài cấp thư mục cha tìm `bin\bash.exe/usr\bin\bash.exe`; (2) nếu vẫn phải fallback quét PATH trực tiếp, PHẢI loại trừ mọi path chứa `WindowsApps` trước khi chấp nhận.

```
private static string? FindGitBashViaGitExeLocation()
{
    var pathEnv = Environment.GetEnvironmentVariable("PATH") ?? string.Empty;
    foreach (var dir in pathEnv.Split(Path.PathSeparator))
    {
        var gitExe = Path.Combine(dir, "git.exe");
        if (!File.Exists(gitExe)) continue;

        var current = new DirectoryInfo(dir);
        for (int i = 0; i < 3 && current is not null; i++, current = current.Parent)
        {
            var b1 = Path.Combine(current.FullName, "bin", "bash.exe");
            if (File.Exists(b1)) return b1;
            var b2 = Path.Combine(current.FullName, "usr", "bin", "bash.exe");
            if (File.Exists(b2)) return b2;
        }
    }
    return null;
}

// Fallback quét PATH — BẮT BUỘC loại trừ WindowsApps
foreach (var dir in pathEnv.Split(Path.PathSeparator))
{
    if (dir.Contains("WindowsApps", StringComparison.OrdinalIgnoreCase)) continue;
    var candidate = Path.Combine(dir, "bash.exe");
    if (File.Exists(candidate)) return candidate;
}
```

1. Không tin tưởng tuyệt đối vào 1-2 hardcoded path cho các tool có thể cài custom location (Git, Python, Node...).
2. Khi cần tìm executable phụ trợ của 1 tool (`bash.exe` của Git), ưu tiên derive từ executable CHÍNH của tool đó (`git.exe`) đã biết chắc có trong PATH, thay vì đoán tên file phụ trợ rồi quét PATH ngây thơ.
3. Viết unit test xác nhận path trả về KHÔNG chứa "WindowsApps" — regression test rõ, phát hiện ngay nếu logic detect bị đổi sai sau này.

Áp dụng lại (How to reuse)

- Khi thấy lỗi `cd: /X/...: No such file or directory` dù path Windows gốc THẬT SỰ tồn tại, và code có logic tự detect + gọi `bash.exe/sh` — nghi ngay việc detect trúng nhầm executable (WSL stub, hoặc bash khác không phải MSYS) TRƯỚC khi nghi logic convert path.
- Kiểm tra nhanh: `where.exe bash` (PowerShell) — nếu 1 trong các kết quả nằm trong `...\WindowsApps\...` VÀ máy có cài WSL, xác nhận bằng `Get-Item <path> | Select Target` hoặc tương đương xem nó có phải symlink trỏ `wsl.exe` không.
- Khi code cần derive path phụ trợ của 1 CLI tool, tìm bằng cách derive từ executable chính (đáng tin cậy hơn) thay vì đoán tên file rồi quét PATH.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Hành vi này CHỈ xảy ra trên máy có bật tính năng Windows Subsystem for Linux — máy chưa bật WSL sẽ không có file `WindowsApps\bash.exe`, nên bug này "works on my machine" (máy dev khác không bật WSL) nhưng fail trên máy khác — rất khó reproduce nếu không biết máy đích có WSL hay không.
- ⚠ `git.exe` có thể nằm ở `<GitRoot>\cmd\`, `<GitRoot>\bin\`, hoặc `<GitRoot>\mingw64\bin\` tùy version/cách cài Git — code derive path phải dò nhiều cấp thư mục cha (không giả định cấu trúc cố định).
- ⚠ Đừng nhầm bug này với vấn đề "Git Bash gọi wsl.exe cần MSYS_NO_PATHCONV" (đã ghi trong lesson khác [wsl-hybrid-build-windows-projectref-msys-pathconv.md](#)) — đó là vấn đề Git Bash TỰ GỌI wsl.exe và path bị dịch sai; bug này là code TỰ detect nhầm executable ngay từ đầu, chưa liên quan gì đến wsl.exe cả.

Tham chiếu

- Project liên quan: `E:\KZTEK\Code_Git\7.LinuxDeployTool`
(`Kztek.LinuxDeployTool.Services.BashRunner.FindGitBash`)
- Liên quan: [dotnet-general/wsl-hybrid-build-windows-projectref-msys-pathconv.md](#) (vấn đề path conversion khác, không phải cùng gốc)